

📁 OOAD Ch9 Problem A：Blockbuster DVD 系統分析解答

主題重點： 針對 [Ch9 筆記](#) 中的 Problem A（百視達 DVD 租片系統），整理出符合課堂分析邏輯的 class 設計、[Rental](#) 中介類別、[print_statement\(\)](#) 虛擬碼，以及 [main\(\)](#) 展示流程。

📌 解題核心觀念

根據課堂提到的 正規化 與 展開多對多關聯 的概念：

- 不能直接把「電影」塞進 [Customer](#) 的清單就結束
- 因為「租借」這件事本身還有自己的資料，例如：
 - 租借天數
 - 金額
 - 積點
- 因此 [Customer](#) 與 [Movie](#) 之間，不應直接用簡單多對多連接
- 中間應該建立一個新的關聯物件：[Rental](#)

也就是說，這題的核心不是只會算錢，而是要先把：

Customer — Rental — Movie

這個結構分析對。

✂ 類別設計總覽

此題至少可以分析出三個主要 classes：

1. [Movie](#)
2. [Rental](#)
3. [Customer](#)

如果未來想做得更有彈性，也可以再往下拆出像 [Strategy](#) 之類的計價策略類別；但依照目前題目要求，三個核心類別就已足夠完成需求。

📁 1. Movie 類別

職責

- 儲存電影基本資料
- 根據片型決定租金計算方式
- 根據片型回傳常客積點

Attributes

```
string title
int price_code
const int NEW_RELEASE = 1
const int REGULAR = 2
const int CHILDREN = 3
```

Methods

```
getCharge(int daysRented)
getFrequentRenterPoints()
```

設計說明

[Movie](#) 應該知道自己的片型，因此可以自己決定：

- 怎麼算租金
- 可以拿幾點積點

這樣能避免把所有計價規則都塞到 [Customer](#) 裡，讓 [Customer](#) 變成過胖類別。

📁 2. Rental 類別

職責

- 代表一次實際的租借紀錄
- 負責連接 [Customer](#) 與 [Movie](#)
- 保存該次租借特有的資料（例如租借天數）

Attributes

```
Movie movie
int daysRented
```

Methods

```
getCharge()
getPoints()
```

設計說明

Rental 是這題最重要的分析點。

如果沒有 **Rental**：

- 就沒地方記錄每次租借的天數
- 未來若要加强租借日期、還片日期、逾期狀態，也會沒地方放

因此 **Rental** 是典型的：

- 展開多對多關聯之後產生的中介類別
- 也是課堂上 KRB Step 4 的直接應用

3. Customer 類別

職責

- 代表租片顧客
- 保存顧客所有租借紀錄
- 負責輸出該顧客的報表

Attributes

```
string name
Vector<Rental> rentals
```

Methods

```
void addRental(Rental arg)
void print_statement()
```

為什麼 **print_statement()** 放在 Customer？

因為這份報表的内容是：

- 某位客戶過去租過哪些片
- 這位客戶的總消費是多少
- 這位客戶累積了多少積點

所以最自然的歸屬位置就是 **Customer**。

也就是說，這不是在列印單一電影資訊，而是在列印：

某位顧客的租借摘要

因此把 **print_statement()** 放在 **Customer** 類別是合理的。

print_statement() 虛擬碼

```
class Customer {
    string name
    Vector<Rental> rentals

    void print_statement() {
        double totalAmount = 0.0;
        double frequentRenterPoints = 0.0;

        print "客戶: " + name + " 的租片報表\n";

        for each (Rental r in rentals) {
            double thisAmount = r.getCharge();
            frequentRenterPoints += r.getPoints();

            print r.movie.title + " : $" + thisAmount + "\n";

            totalAmount += thisAmount;
        }

        print "總消費金額 = $" + totalAmount + "\n";
        print "常客積點總數 = " + frequentRenterPoints + " 點\n";
    }
}
```

print_statement() 的邏輯拆解

這個 method 的流程其實很清楚：

1. 初始化總金額與總積點

```
double totalAmount = 0.0;
double frequentRenterPoints = 0.0;
```

2. 逐筆走訪 rentals

每一筆 **Rental**：

- 呼叫 **r.getCharge()** 算出該次租借金額

- 呼叫 `r.getPoints()` 算出該次積點

3. 印出片名與該筆費用

```
print r.movie.title + " : $" + thisAmount;
```

4. 加總總金額與總積點

把每筆結果累加起來。

5. 最後輸出總結

輸出：

- 總消費金額
- 常客積點總數

這裡也可以看出物件分工：

- `Customer` 負責整理整份報表
- `Rental` 負責每筆租借紀錄
- `Movie` 負責片型規則

這樣的責任切分比把所有邏輯硬塞在同一個 class 更合理。

main() 展示流程

根據題目要求，`main()` 需要完成：

- 建立 2 位 customer
- 建立 3 部電影
- 建立租借關係
- 印出報表

虛擬碼

```
main() {
    Customer april = new Customer("April");
    Customer joe = new Customer("Joe");

    Movie wonderWoman = new Movie("神力女超人", Movie.REGULAR);
    Movie justiceLeague = new Movie("正義聯盟", Movie.NEW_RELEASE);
    Movie thor3 = new Movie("雷神索爾3", Movie.NEW_RELEASE);

    april.addRental(new Rental(wonderWoman, 3));
    joe.addRental(new Rental(wonderWoman, 2));
    april.addRental(new Rental(justiceLeague, 5));
    joe.addRental(new Rental(thor3, 1));

    april.print_statement();
    joe.print_statement();
}
```

✦ 題目要求與分析對應

題目要求 1: 影片計價邏輯

這部分應由 `Movie` 負責依照 `price_code` 判斷：

- 新片：\$3，租 1 天，逾期每天 +2
- 舊片：\$2，租 3 天，逾期每天 +1
- 兒童片：\$2，租 5 天，逾期每天 +1

題目要求 2: 常客積點

同樣由 `Movie` 依片型回傳：

- 新片：2.0
- 舊片：1.0
- 兒童片：0.5

題目要求 3: 列印報表

由 `Customer.print_statement()` 完成：

- 過去租過的片子清單
- 總金額
- 積點總數

延伸設計思考

老師在題目中有特別提醒：

常客積點優惠經常會改，需保留彈性。

這代表如果真的要將系統設計得更好，未來可以進一步考慮：

- 把計價規則從 `Movie` 中拆出來
- 把積點規則也策略化

- 讓不同片型透過不同策略物件處理

例如可能發展成：

- `PriceStrategy`
- `RegularPrice`
- `NewReleasePrice`
- `ChildrenPrice`

這樣未來如果規則改變，比較不需要一直修改同一堆 `if/else`。

也因此，這題除了是類別分析題，某種程度上也在暗示：

- 哪些規則是穩定的
- 哪些規則是常變的
- 常變的地方應該特別保留彈性

✔ 解題總結

1. `Customer` 和 `Movie` 之間不能直接簡單連接，應透過 `Rental` 中介類別。
2. `Rental` 是展開多對多關聯的結果，也是正規化思維的實作。
3. `Movie` 負責片型對應的金額與積點規則。
4. `Customer` 負責保存租借清單並輸出報表。
5. `print_statement()` 最合理的歸屬位置是 `Customer`。
6. 若未來要讓設計更有彈性，可考慮把計價規則進一步抽成策略類別。

🔗 Related Notes

- [School/OOAD/CH9_How_to_Find_Classes_and_Objects](#)
- [School/OOAD/CH8_OOAD_ERD_and_Normalization](#)
-  [School/OOAD_Object-Oriented Analysis and Design](#)